Table of Contents

Cover page

[Institution name]

Department of Computer Science and Engineering

Operating Systems Mini Project

msh: A Minimal UNIX Shell

Course: Operating Systems (T7998), Credit 4

Year / Semester: 2026 / III

Batch: 2025–29

Faculty guide: Dr. Bibhuprasad Sahu

Academic year: 2026–27

Submitted by

[Member 1 name] [PRN / roll number]

[Member 2 name] [PRN / roll number]

[Member 3 name] [PRN / roll number]

[Member 4 name] [PRN / roll number]

Abstract

This project presents msh, a small UNIX shell written in C and used from the command line. A shell is the program that reads a typed line and asks the operating system to run it. msh does that with fork, execvp, waitpid, pipe, open, and dup2. It does not call system(), and it does not pass the line to another shell. An external command is a new process. fork creates the child, execvp replaces the child with the program, and waitpid collects the child so it does not remain a zombie. Commands that must change the shell, such as cd and exit, run in the shell process. A pipeline joins children with an anonymous pipe, the inter-process

communication tool listed with shared memory and message passing. Redirection opens a file and uses dup2 so the file becomes standard input or standard output. The shell stops at those ideas. It does not implement job control or a shell programming language, and it does not implement the scheduling, deadlock, paging, or disk algorithms from the rest of the course. Thirty-three functional tests cover ordinary commands, pipelines, redirection, and invalid input. On the measurement machine a builtin pwd cost about 0.020 ms, an external true cost about 1.103 ms, and a two-process pipeline cost about 1.843 ms.

1. Introduction

A shell is an ordinary program. It prints a prompt, reads a line, and asks the kernel to run the program named on that line. When a user types ls, the shell does not contain the code of ls. It creates a new process and loads ls into that process. That is the whole idea this project is built to show.

The course introduces an operating system as a layer that offers services through system calls [3]. Process management is one of those services: a process is created, it runs, and it terminates. Inter-process communication includes pipes as well as shared memory and message passing. File management includes opening a file and reading or writing it. msh uses only those three ideas. fork and execvp create and load a process. waitpid collects it. pipe connects two processes. open and dup2 attach a file to a process. Nothing else in the program is required to explain it.

UNIX stated this design early. Ritchie and Thompson described processes that are cheap to create, and a single read/write interface for files, devices, and pipes [1]. Bourne’s shell, published in 1978, is both a command language and a programming language on top of that interface [2]. This project takes the command half only: find a program, start it, connect it, and wait. The programming-language half, with if, while, and variables, is a different project.

The interface is the command line because that is what a shell is. The program is named msh. Make builds it from four C files. Python is used only to run tests. The shell links only the C library.

2. Problem statement and objectives

2.1 Problem statement

A sentence such as “fork creates a process and exec loads a program” is easy to memorize and hard to picture. A pipe is the same. It is a kernel buffer with two ends. One process writes one end. Another process reads the other end. The sentence becomes concrete only when both processes exist and the bytes move. Redirection is the same idea pointed at a file instead of a pipe.

The problem is to build a command-line shell that performs these steps with its own system calls, and to show that the result is correct on ordinary input and on input that should fail. A missing program, a missing file, or a line that is only a pipe character must print an error and leave the shell running.

The shell must also be explainable in a short viva. Every call that creates a process, a pipe, or an open file sits in a file whose name matches the job.

2.2 Objectives

1. Parse a command line into words, quotes, a pipeline, and input, output, and append redirection.
2. Run an external program with fork and execvp, and collect it with waitpid.
3. Run cd, pwd, exit, and help in the shell process when they are a simple command.
4. Connect pipeline stages with anonymous pipes, and close the unused ends so a reader sees end of file.
5. Apply <, >, and >> with open and dup2, including a redirect that replaces one end of a pipe.
6. Accept a script on standard input with no prompt, which is what the tests use.
7. Include invalid input in the tests, and measure a builtin, an external command, and a short pipeline.

2.3 Scope

The shell covers process creation, pipes, and file redirection. It remembers the exit status of the last command and prints it when the user types $?. That number is the value waitpid returned.

It does not implement background jobs, process groups, or Ctrl+Z. It does not implement &&, ||, ;, or 2>. It does not implement globbing, command substitution, or general variables. Those are shell-language features. Scheduling, semaphores, deadlocks, paging, and disk scheduling are in the course and are not this topic. Leaving them out keeps the viva on the calls the program actually makes.

2.4 Expected outcome

The outcome is a working binary, a test suite, a sample session, and the measurements in Section 7. A person at the prompt should be able to run a command, a pipeline, and a redirection, and should be able to name the system call that did the work.

2.5 Alignment with the prescribed topic

Topic 21 asks for a command-line interpreter that supports commands and process creation with fork() and exec() on Linux, written in C. msh is that program. The child calls execvp, which searches PATH, so a typed name such as ls becomes a new program. The parent waits with waitpid. cd and exit stay in the parent, because a child cannot change its parent's directory or end the shell by exiting.

Pipes and redirection are included because the guidelines ask for a working command line, invalid-input tests, and a measurement, and because a pipe is the inter-process communication example in the process module. Both use the same process and file-descriptor model. Neither is handed to another shell.

3. Literature and existing systems

3.1 The process and the pipe

Silberschatz, Galvin, and Gagne describe a process as a program in execution, and the system calls that create a process and wait for it [3]. Tanenbaum and Bos describe the same split: fork duplicates the caller, and a member of the exec family replaces the new process image [4]. Bach describes how the UNIX kernel implements those calls [6]. msh follows that split and does not reimplement the kernel. It calls the C library, which calls the kernel.

A pipe is a kernel buffer. The writer blocks when the buffer is full. The reader blocks when it is empty. The reader receives end of file only when every write end is closed. That last rule is

why the shell must close the pipe ends it does not use. Stallings describes pipes as one of the standard inter-process communication tools, beside shared memory and message passing [7].

3.2 What a production shell adds

Bourne’s shell finds programs, redirects input and output, and starts processes that communicate through pipes [2]. It is also a programming language, with variables and control flow. Table 1 records the line between that product and this project. A yes means the feature is present for the cases in the test suite.

Table 1. Feature comparison

Feature Bourne / POSIX shell msh External command via PATH yes yes Builtin cd and exit yes yes Pipeline of n commands yes yes, up to 32 stages Input and output redirection yes yes, < > and >> Exit status of the last command yes yes, printed by $? Background jobs and Ctrl+Z yes no &&, ||, ;, and 2> yes no Globbing and variables yes no Shell scripts with if/while yes no

The calls themselves are specified by POSIX [5]. The sources are intended to build with clang or gcc. The timings in Section 7 were taken on one machine and should be repeated on the machine used for the demo.

3.3 Why system() is the wrong tool

The C library provides system(), which passes a string to the platform shell. Using it would make msh a prompt in front of the shell this project is supposed to be. The status, the pipeline, and the files would belong to that other shell. The parser builds an argv array and the child passes that array to execvp. Quotes are handled in msh because the kernel does not know that two words were inside one pair of quotes.

4. Requirements and tools

4.1 Functional requirements

R1. The shell reads lines from standard input until end of file or until exit. R2. On a terminal it prints a prompt of the form msh:$ on standard error. The directory comes from getcwd. R3. An external command runs as a child. If execvp cannot find it, the child exits 127 and the shell prints “command not found”. R4. cd, pwd, exit, and help are builtins. A simple builtin

runs in the shell. cd changes the shell’s working directory. exit ends the shell. R5. A pipeline cmd1 | cmd2 | cmd3 runs each stage in its own process. Standard output of each stage except the last is the standard input of the next. R6. < file opens a file for reading on standard input. > file creates or truncates a file for standard output. >> file appends. A redirection applies to the command it is written on and replaces that stage’s pipe end. R7. $? expands to the status of the previous command when the token is not quoted. R8. Invalid input prints an error and a non-zero status, and the shell accepts another command. The invalid cases are an empty pipeline stage, a redirection with no target, a duplicate redirection, an unclosed quote, an unsupported operator, a missing input file, a directory cd cannot enter, and a non-numeric exit status.

4.2 Non-functional requirements

The source is C11 and is split so that a reader can open the parser without reading the executor. The functions are parse_line, execute_pipeline, and builtin_run. Errors use the prefix msh:. The project builds with warnings enabled. make test needs no network and no extra packages.

4.3 Users and operating environment

The user is a person at a terminal, or a test that writes a script to standard input. The laboratory machines run Fedora. The same sources were built on the development machine named in Section 7. No feature depends on a Linux-only call.

4.4 Tools

The compiler is cc, which is clang or gcc. Make builds the binary. Python 3 runs the tests and the benchmark. The only library is the C library. The editor and the version-control tool are not part of the running program.

4.5 Deliverables

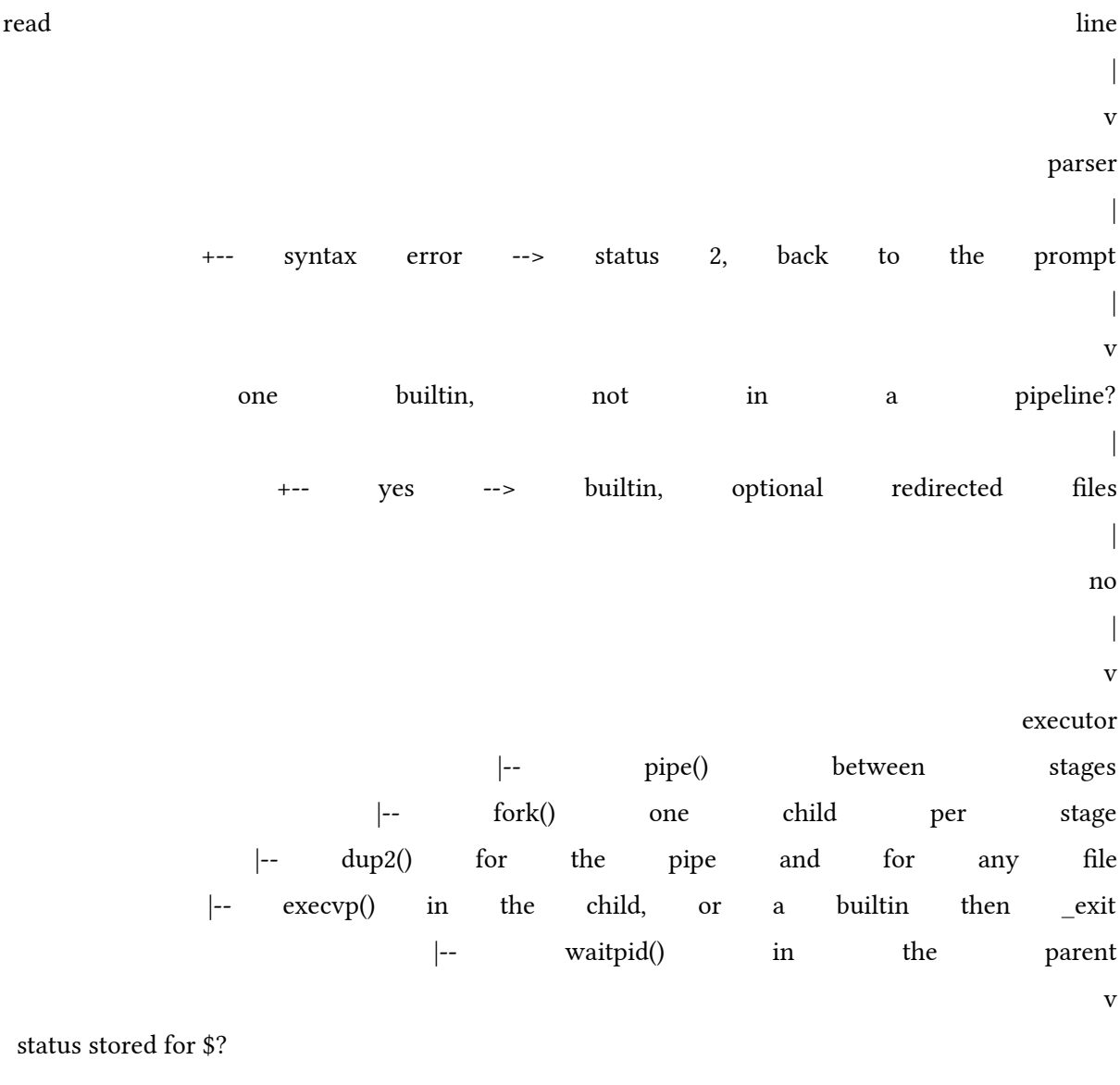
The deliverables are the source, the Makefile, the tests, the sample session, this report, and a public README. The README does not carry roll numbers or course identifiers. Those belong on the cover of the printed report and on the topic form.

5. System design

5.1 Architecture

The main loop reads a line. The parser turns the line into a pipeline or reports a syntax error. The executor either runs one builtin in the shell or forks one child per stage. Figure 1 is that path.

Figure 1. Path of one command line



The parser does not fork. A syntax error therefore cannot leak a process. cd stays out of a careless fork, so a directory change is visible to the next command.

5.2 Data structures

A Command holds a NULL-terminated argv array, an optional input path, an optional output path, and an append flag. A Pipeline holds up to 32 commands. Limits are fixed: 128

arguments per command and 32 stages per pipeline. Hitting a limit prints an error and does not overflow the array.

The parser allocates each word with malloc. The main loop frees the pipeline after the command. There is no job table, because this shell does not run commands in the background. The parent always waits, so a finished child is reaped immediately and does not become a zombie.

5.3 Parsing

The parser scans left to right. Whitespace separates words. A word runs until whitespace or one of |, <, >, &, and ;, except inside quotes. Adjacent quoted pieces form one word, so ‘a’“b” is the argument ab. An unclosed quote is a syntax error. A # outside quotes starts a comment.

The operator | finishes the current command and starts the next stage. A leading pipe, a trailing pipe, or an empty stage is a syntax error. < consumes the next word as the input path. > and >> consume the output path. A second redirection of the same stream is a syntax error. The operators &, ;, &&, ||, and 2> are rejected with “operator not supported”.

After the scan, every stage must have a program name. A line that is only > outfile is rejected. The file is not created, because the shell never reaches open.

Unquoted $? is replaced with the decimal status of the previous command. Quoted $? stays the two characters dollar and question mark.

5.4 Execution and file descriptors

If the pipeline has one command and that command is a builtin, the executor runs it in the shell. Redirection saves the old standard input or output with dup, points the descriptor at the file with dup2, runs the builtin, flushes stdio, and restores the saved descriptor. The flush is required because the C library buffers a FILE object. Writing the buffer after the descriptor is restored would send the output to the terminal instead of the file. The same flush is required before a builtin in a child calls _exit, because _exit does not flush.

Every other pipeline is a sequence of forks. The parent creates n-1 pipes for n stages before forking, so every child inherits the descriptors it needs. Child i copies the read end of the previous pipe onto standard input, and the write end of the next pipe onto standard output. It

then closes every pipe descriptor that is not 0 or 1. If a write end stayed open, a reader such as wc would wait forever.

File redirection is applied after the pipe and replaces it for that stage. So printf ‘x’ > out | cat writes x into out, and cat reads an empty pipe. printf ‘b’ | sort > out leaves sort writing to out. A failed open prints the path and the system error, and the child exits 1. The parent still waits.

The child then runs a builtin and calls _exit, or calls execvp. A missing program exits 127. A program that cannot be executed exits 126. The parent does not treat an exec failure as its own error, because the failure happens in the child.

The status of a pipeline is the status of the last stage. A child that dies from signal N is reported as 128+N. Earlier children are still waited for, so they do not become zombies.

6. Implementation

6.1 Files

src/main.c reads lines, prints the prompt, and decides when to exit. src/parser.c recognizes words, quotes, pipes, and redirection. src/execute.c creates processes and file descriptors. src/builtin.c implements cd, pwd, exit, and help. There is no fifth module. Background jobs were removed so that the program would not depend on process groups or signals.

6.2 Main loop

getline reads a line. The trailing newline is stripped. An empty line or a comment returns to the prompt without changing the status. parse_line either fills a Pipeline or prints a syntax error, in which case the status becomes 2. execute_pipeline returns the status that $? should see. If exit was requested, the loop ends. End of file is the same path. A script whose last command is false makes msh itself exit 1.

6.3 Executor details

dup2 of a pipe end onto descriptor 0 or 1 is followed by a close of the other copies. The close loop does not close descriptor 0 or 1 if a pipe end is already that descriptor. This avoids closing standard input because a pipe happened to be allocated on descriptor 0.

chdir is the whole of cd. getcwd after a successful chdir supplies the path stored in PWD. On the test machine, cd /tmp followed by pwd prints /private/tmp, because /tmp is a symbolic link. The tests compare against pwd -P for that reason.

exit does not call the C library exit() from the parent builtin. It sets a flag that the main loop sees after redirection has been restored. In a child, the builtin calls _exit so that the parent’s handlers do not run.

execvp searches PATH. No command is executed by passing the raw line to /bin/sh.

6.4 One line, call by call

The line `printf 'b\na\n' | sort > out` is enough to show every call the viva is likely to ask about. The parser makes two commands. The first has the arguments printf, b, and a, with the newlines kept inside the quotes. The second has the argument sort and the output path out.

The parent calls pipe once. That returns a read descriptor and a write descriptor. It then calls fork twice. Both children inherit both descriptors.

The first child uses dup2 to make the write end its standard output. It closes the read end and the extra copy of the write end. It has no file redirection. It calls execvp on printf. printf writes the two lines into the pipe and exits.

The second child uses dup2 to make the read end its standard input. It closes the write end. It then opens out with open, using the create and truncate flags, and uses dup2 to make that file its standard output. The pipe’s read end is still its standard input, so sort reads from printf and writes to the file. It calls execvp on sort.

The parent closes both of its pipe ends and calls waitpid for each child. The status of sort is the status of the pipeline. If that status is 0, a later `echo $?` prints 0. If the parent had left its write end open, sort would not see end of file and the shell would look stuck. The close is part of making the pipe work, not an optional cleanup.

7. Testing and results

7.1 Method

Functional tests are a Python unittest module. Each test starts ./msh, writes a script to its standard input, and compares standard output, standard error, and the process exit code. The

shell is not a terminal in these tests, so there is no prompt. Each test uses a fresh process. Temporary files live in a temporary directory that the test deletes.

7.2 Functional results

All 33 functional tests passed on the measurement machine after a clean build. Table 2 groups them.

Table 2. Functional coverage

Group What was checked Tests Simple execution echo, quotes, adjacent quotes, comments, help, quoted $? 6 Directory builtins pwd, cd, cd -, missing path, file path, too many 8 arguments, HOME unset, cd inside a pipeline Redirection create, append, input, empty input, missing input, 6 builtin pwd to a file, a redirect that replaces a pipe Pipelines sort, three stages, 31-stage cat chain, output file, 6 status of the last stage, builtin pwd through a pipe Errors command not found (127), syntax errors (status 2), 4 duplicate redirection, bad exit argument Process lifetime exit code, exit of the previous status, EOF status 3

The syntax errors include a trailing pipe, a leading pipe, a redirection with no target, an unclosed quote, &, ;, 2>, and a redirection with no command. Each leaves the shell able to print $? as 2.

7.3 Measurements

The benchmark runs each command many times inside one msh process and reports the median of five trials. The machine was Darwin 27.0.0 on arm64, not the Fedora laboratory image. The numbers should be measured again on the demo machine. They are included to show the shape of the cost, not a universal constant.

Table 3. Median cost per command

Command Median time pwd redirected to /dev/null 0.020 ms /usr/bin/true 1.103 ms echo hello | cat, output discarded 1.843 ms

pwd is a builtin, so it does not call fork. true is an external program that does no other work, so its time is mostly process creation. The pipeline creates two processes and one pipe, so it costs more than one external command. The ratio is the reason a shell has builtins. Creating a process to print the current directory would be about fifty times more expensive on this machine than calling getcwd in the shell.

A separate probe writes to a non-blocking pipe until the write would block. The capacity on this kernel was 65536 bytes. msh does not contain that number. The probe only shows that the kernel, not the shell, buffers the pipe. The writer runs until the buffer is full. If the reader is running, it frees space. If the reader is not running, the writer blocks inside write.

8. Discussion

The choice that matters most is where a builtin runs. Running every builtin in a child makes cd a no-op, because the child exits and takes the new directory with it. Running a builtin in the parent when it is part of a pipeline makes pwd | cat write to the terminal, and makes cd /tmp | cat change the shell as a side effect. The rule used here is: run in the parent only when the builtin is the whole pipeline. The tests cover both sides. The stdio flush belongs to the same rule. Redirecting descriptor 1 does not redirect bytes that printf has stored and not written.

The second choice is to close unused pipe ends. A pipeline that forgets this looks hung. wc waits for an end of file that never arrives, because the shell or another stage still holds a write end. The fix is a close, not a larger buffer in user space.

The third choice is to stop at the course material this topic needs. Job control needs process groups and the terminal's foreground group. List operators such as && are shell syntax, not a system call. Both are easy to add and hard to explain if the speaker has not studied them. The sample session is still a real session: a quoted echo, a directory change, a pipeline into wc, a missing program, and a syntax error.

The measurements should be cited as observations on one machine. A different kernel can report a different pipe capacity. The right sentence is "on the test machine, the probe observed 65536 bytes."

9. Conclusion and future scope

9.1 Conclusion

msh is a command-line shell that creates processes with fork and execvp, waits for them with waitpid, connects them with pipes, and points them at files with dup2. Builtins that change the shell run in the shell. Invalid lines produce a status and an error and leave the shell running. The tests lock these behaviors in. The timings show why a builtin exists: creating a

process that does nothing costs about 1.1 ms on the test machine, and avoiding that creation costs about 0.02 ms.

The project meets the objectives in Section 2. The parts that were left out are named in Section 2.3. They are left out so that the program stays inside process creation, pipes, and files.

9.2 Future scope

A later version could add background jobs, which means process groups and a job table, or the list operators &&, ||, and ;. Neither change replaces the process model already here. A larger and different project would be variables and the if/while constructs Bourne described [2]. Scheduling, deadlocks, paging, and disk scheduling remain separate course topics and are not missing pieces of this shell.

References

[1] D. M. Ritchie and K. Thompson, “The UNIX time-sharing system,” Commun. ACM, vol. 17, no. 7, pp. 365–375, Jul. 1974, doi: 10.1145/361011.361061.

[2] S. R. Bourne, “UNIX time-sharing system: The UNIX shell,” Bell Syst. Tech. J., vol. 57, no. 6, pp. 1971–1990, Jul.–Aug. 1978, doi: 10.1002/j.1538-7305.1978.tb02139.x.

[3] A. Silberschatz, P. B. Galvin, and G. Gagne, Operating System Concepts, 9th ed. Hoboken, NJ, USA: Wiley, 2012.

[4] A. S. Tanenbaum and H. Bos, Modern Operating Systems, 4th ed. Upper Saddle River, NJ, USA: Pearson, 2015.

[5] IEEE and The Open Group, The Open Group Base Specifications Issue 7 (IEEE Std 1003.1-2017), 2018. [Online]. Available: https://pubs.opengroup.org/onlinepubs/9699919799/

[6] M. J. Bach, The Design of the UNIX Operating System. Englewood Cliffs, NJ, USA: Prentice Hall, 1986.

[7] W. Stallings, Operating Systems: Internals and Design Principles, 8th ed. Upper Saddle River, NJ, USA: Prentice Hall, 2015.

The course prescribes Silberschatz, Galvin, and Gagne, 9th edition [3]; Stallings, 8th edition [7]; Tanenbaum and Bos, 4th edition [4]; Godbole, 2nd edition; and Bach [6]. Godbole is a prescribed textbook for the same process and file material and is not quoted above.

Appendix A. Sample session

The transcript in samples/session.txt is the session to attach if the printed report needs a figure of the screen. It includes help, a quoted echo, cd to /tmp, cd -, append redirection, a three-stage pipeline, a failing command, an unknown command, and a syntax error.

Appendix B. Test catalogue

The executable catalogue is tests/test_shell.py. Running make test is the check this appendix describes. The groups are the rows of Table 2. The tests do not require a network, a display, or a privileged account.

Appendix C. Module map for the viva

Question: what is a shell? Answer: a program that reads a command line and asks the operating system to run it. msh is that program. It is not the kernel.

Question: where is a pipeline created? Answer: parse_line in src/parser.c fills a Pipeline. It does not create processes.

Question: where is a process created? Answer: execute_pipeline in src/execute.c calls fork. The child calls execvp.

Question: why does cd work? Answer: a simple cd runs in the shell process and calls chdir. The same cd inside a pipeline runs in the child, and the shell directory does not change. A test checks that.

Question: what is a pipe? Answer: a kernel buffer. pipe creates two descriptors. dup2 makes one the writer’s standard output and the other the reader’s standard input. Unused ends are closed so the reader sees end of file.

Question: what does redirection do? Answer: open opens the file. dup2 makes that descriptor standard input or standard output. The command then reads or writes the file without knowing the file’s name.

Question: what is $? Answer: the exit status of the previous command. waitpid collected it. The parser substitutes the number when $? is not quoted.